TrueSaya Product Overview

TrueSaya Product is a RESTful API platform designed to support automatic microservices build-up for software teams.

> It is best suited for teams that want to expose services through APIs while simplifying how microservices are created, connected, and managed.

1. Microservice Architecture: Basic Concepts

- **Microservice**: Small, independently deployable service focused on one business capability.
- **Business Capability**: Services are divided by domain, e.g., Customer, Order, Payment, Notification.
- **Independent Deployment**: Each service can be built, tested, released, and scaled independently.
- **Spring Boot**: Common Java framework for building REST-based microservices.
- **Spring Cloud**: Supports distributed-system capabilities such as configuration, service patterns, and resilience integrations.
- **API Gateway**: Single entry point for clients; handles routing, authentication, rate limiting, etc.
- **Service-to-Service Communication**: REST/HTTP, gRPC, or asynchronous messaging.
- **Event-Driven Communication**: Apache Kafka/RabbitMQ for asynchronous, loosely coupled processing.
- **Database per Service**: Each microservice owns and controls its data.
- **Service Discovery**: Enables services to locate other service instances dynamically.
- **Centralized Configuration**: Externalizes environment-specific application configuration.
- **Resilience**: Timeout, Retry, Circuit Breaker, Bulkhead, Rate Limiting, typically using Resilience4j.
- **Security**: OAuth 2.0, OpenID Connect, JWT, Spring Security.
- **Containerization**: Docker packages services consistently.
- **Orchestration**: Kubernetes manages deployment, scaling, availability, and recovery.
- **Observability**: Centralized logs, metrics, distributed tracing, health checks, and alerting.
- **CI/CD**: Automated build, testing, security scanning, and deployment pipeline.

---

2. Microservices vs Monolithic Architecture

Area	Monolithic	Microservices
Architecture	Single application	Multiple independent services
Deployment	Entire application deployed together	Services deployed independently
Scaling	Scale entire application	Scale individual services

Coupling	Generally higher	Generally lower
Technology	Usually one technology stack	Can support different technology stacks
Database	Often shared/central DB	Prefer database per service
Failure	Failure can impact whole application	Failure can often be isolated
Development	Easier initially	Better for large independent teams/domains
Operations	Simpler	More complex
Communication	In-process method calls	Network/API/event communication
Best Fit	Small/medium or simpler systems	Large, complex, evolving enterprise platforms

Microservices: Advantages

- Independent deployment
- Independent scaling
- Better fault isolation
- Strong domain/business boundaries
- Supports parallel development by multiple teams
- Technology flexibility
- Faster independent releases
- Well suited to cloud/container environments

Microservices: Disadvantages

- Higher architectural complexity
- Network latency and communication failures
- Distributed transaction complexity
- Data consistency challenges
- More difficult debugging/troubleshooting
- Requires mature DevOps/CI/CD
- Higher infrastructure and operational overhead
- More complex monitoring, logging, tracing, and security

Monolith: Advantages

- Simpler architecture
- Easier development initially
- Easier local testing/debugging
- Simpler deployment
- Easier database transactions
- Lower infrastructure/operational complexity

Monolith: Disadvantages

- Tight coupling as application grows
- Entire application requires deployment
- Harder to scale individual functionality
- Larger codebase becomes difficult to maintain
- Changes can have wider regression impact
- Slower releases for large applications

---

3. Microservice Design Patterns and Principles: List Only

Design Patterns

1. API Gateway Pattern
2. Backend for Frontend (BFF) Pattern
3. Database per Service Pattern
4. Saga Pattern
5. CQRS Pattern
6. Event Sourcing Pattern
7. Circuit Breaker Pattern
8. Retry Pattern
9. Timeout Pattern
10. Bulkhead Pattern
11. Service Discovery Pattern
12. Externalized Configuration Pattern
13. Sidecar Pattern
14. Strangler Fig Pattern
15. Anti-Corruption Layer Pattern
16. Aggregator Pattern
17. API Composition Pattern
18. Transactional Outbox Pattern
19. Event-Driven Architecture Pattern

20. Publish-Subscribe Pattern
21. Competing Consumers Pattern
22. Idempotent Consumer Pattern
23. Dead Letter Queue Pattern
24. Health Check Pattern
25. Ambassador Pattern
26. Service Mesh Pattern
27. Gateway Routing Pattern
28. Gateway Aggregation Pattern
29. Gateway Offloading Pattern
30. Consumer-Driven Contract Pattern

Principles

1. Single Responsibility Principle
2. Separation of Concerns
3. Loose Coupling
4. High Cohesion
5. Domain-Driven Design
6. Bounded Context
7. Decentralized Data Management
8. API-First Design
9. Contract-First Design
10. Independent Deployability
11. Independent Scalability
12. Automation
13. Infrastructure as Code
14. Design for Failure
15. Fault Isolation
16. Fail Fast
17. Resilience
18. Idempotency
19. Eventual Consistency
20. Observability
21. Security by Design
22. Zero Trust
23. Statelessness

24. Backward Compatibility

25. Continuous Integration/Continuous Delivery

In one line: A well-designed Java/Spring Boot microservices system aims for **small business-focused services + loose coupling + independent deployment + decentralized data + resilient communication + strong observability and automation**.

Copilot said:

Below is a **short, concise, point-to-point explanation** of every Design Pattern and Principle listed in your uploaded Ritesh_Design_Patterns_List, using one consistent example: an **E-Commerce Shopping Cart Platform built with Java + Spring Boot microservices**. [Ritesh_Des..._List.docx | Word]

Assume the platform contains **Product, Cart, Inventory, Order, Payment, Customer, Shipping, and Notification microservices**, with Spring Boot, REST APIs, Kafka, Redis, PostgreSQL, Docker, and Kubernetes as representative technologies.

1. Microservice Design Patterns

1. API Gateway Pattern

- **Meaning:** Single entry point for all client requests.
- **Purpose:** Routing, authentication, rate limiting and cross-cutting concerns.
- **E-Commerce Example:** `/cart/**` routes to Cart Service, `/orders/**` to Order Service.
- **Java Implementation:** Spring Cloud Gateway + Spring Security.

2. Backend for Frontend (BFF) Pattern

- **Meaning:** Separate backend/API layer customized for each frontend.
- **Purpose:** Avoid one generic API serving very different client requirements.
- **Example:** Web-BFF returns detailed Product + Cart data, while Mobile-BFF returns lightweight responses.
- **Implementation:** Separate Spring Boot Web-BFF and Mobile-BFF services.

3. Database per Service Pattern

- **Meaning:** Every microservice owns its database/schema.
- **Purpose:** Prevent tight data-level coupling.
- **Example:** Cart Service owns Cart DB; Order Service owns Order DB; Payment Service owns Payment DB.
- **Rule:** Order Service must not directly query Cart Service's database.

4. Saga Pattern

- **Meaning:** Manages distributed business transactions using multiple local transactions.
- **Purpose:** Maintain consistency without one global database transaction.

- **Example:** `Create Order → Reserve Inventory → Take Payment → Arrange Shipping`.
- **Failure:** If Payment fails, compensate by releasing reserved inventory and cancelling the order.
- **Implementation:** Kafka events or Saga Orchestrator.

5. CQRS Pattern

- **Meaning:** Separate **Commands/Writes** from **Queries/Reads**.
- **Purpose:** Optimize read and write workloads independently.
- **Example:** `AddItemToCart` modifies cart; `GetCartDetails` reads an optimized cart view.

6. Event Sourcing Pattern

- **Meaning:** Store state changes as events instead of only storing final state.
- **Example:** `CartCreated → ItemAdded → QuantityChanged → ItemRemoved`.
- **Benefit:** Complete history can reconstruct Cart state.

7. Circuit Breaker Pattern

- **Meaning:** Stop repeatedly calling a failing dependent service.
- **Example:** Cart Service temporarily stops calling an unavailable Recommendation Service.
- **Implementation:** Resilience4j `CircuitBreaker`.

8. Retry Pattern

- **Meaning:** Retry temporary/transient failures.
- **Example:** Order Service retries an Inventory Service call after a temporary network failure.
- **Implementation:** Resilience4j Retry with controlled backoff.

9. Timeout Pattern

- **Meaning:** Stop waiting after a configured time.
- **Example:** Cart Service does not wait indefinitely for Pricing Service.
- **Benefit:** Prevents threads/resources getting blocked.

10. Bulkhead Pattern

- **Meaning:** Isolate resources so one failure cannot exhaust the whole application.
- **Example:** Payment and Recommendation calls use separate resource/thread pools.
- **Benefit:** Recommendation overload does not exhaust resources needed for checkout.

11. Service Discovery Pattern

- **Meaning:** Services dynamically locate service instances.
- **Example:** Order Service discovers available Inventory Service instances.
- **Implementation:** Kubernetes Service/DNS or a service registry.

12. Externalized Configuration Pattern

- **Meaning:** Configuration is maintained outside application code.
- **Example:** Payment timeout, Kafka brokers and environment URLs differ across DEV/UAT/PROD.
- **Implementation:** Spring configuration + Kubernetes ConfigMaps/Secrets.

13. Sidecar Pattern

- **Meaning:** Supporting functionality runs alongside the application container.
- **Example:** A sidecar handles proxying, telemetry or security functionality beside Cart Service.
- **Benefit:** Keeps infrastructure concerns outside business code.

14. Strangler Fig Pattern

- **Meaning:** Gradually migrate a monolith to microservices.
- **Example:** Extract Cart functionality first, then Product, Order and Payment from an existing e-commerce monolith.
- **Benefit:** Incremental migration instead of risky big-bang replacement.

15. Anti-Corruption Layer Pattern

- **Meaning:** Translation layer protects the new domain model from a legacy/external model.
- **Example:** Payment Service translates a legacy payment system response into the platform's standard `PaymentResult`.

16. Aggregator Pattern

- **Meaning:** Combines information from multiple services into one response.
- **Example:** Shopping Cart Aggregator combines Cart + Product + Price + Inventory.
- **Result:** Client receives one consolidated Cart response.

17. API Composition Pattern

- **Meaning:** Build a response by querying multiple microservices.
- **Example:** Order Details API obtains Order + Payment + Shipping data and composes the result.

18. Transactional Outbox Pattern

- **Meaning:** Business data and an outgoing event are written within the same local database transaction.
- **Example:** Order Service saves `Order` and `OrderCreated` outbox record together.
- **Benefit:** Prevents DB update succeeding while event publication is lost.

19. Event-Driven Architecture Pattern

- **Meaning:** Services communicate through business events.
- **Example:** Order Service publishes `OrderCreated`; Inventory and Notification Services react independently.
- **Implementation:** Apache Kafka.

20. Publish-Subscribe Pattern

- **Meaning:** One event can be consumed by multiple subscribers.
- **Example:** `OrderCreated` may be consumed by Inventory, Analytics and Notification Services.

21. Competing Consumers Pattern

- **Meaning:** Multiple instances consume messages in parallel.
- **Example:** Several Notification Service instances process thousands of order-notification events.
- **Benefit:** Higher throughput and scalability.

22. Idempotent Consumer Pattern

- **Meaning:** Processing the same event multiple times produces the same intended outcome.
- **Example:** Duplicate `PaymentCompleted` events must **not create two orders or two shipments**.
- **Implementation:** Store/check processed Event ID.

23. Dead Letter Queue Pattern

- **Meaning:** Permanently failing messages are moved aside for investigation/reprocessing.
- **Example:** Invalid `OrderCreated` event moves to an Order DLT/DLQ after configured retry attempts.

24. Health Check Pattern

- **Meaning:** Expose application health/readiness information.
- **Example:** Kubernetes verifies whether Cart Service is ready to receive traffic.
- **Implementation:** Spring Boot Actuator `/actuator/health`.

25. Ambassador Pattern

- **Meaning:** Helper/proxy handles outbound network concerns for a service.
- **Example:** Proxy manages secure connections/retries between Payment Service and an external payment provider.

26. Service Mesh Pattern

- **Meaning:** Infrastructure layer manages service-to-service communication.
- **Example:** Cart → Product and Order → Inventory traffic gets mTLS, telemetry and routing outside business code.
- **Implementation:** Istio/Envoy-style service mesh.

27. Gateway Routing Pattern

- **Meaning:** Gateway routes requests to appropriate downstream service.
- **Example:** `/api/cart/** → Cart Service`; `/api/orders/** → Order Service`.

28. Gateway Aggregation Pattern

- **Meaning:** Gateway calls multiple services and combines their results.

- **Example:** `/checkout-summary` combines Cart + Discount + Inventory + Shipping information.

29. Gateway Offloading Pattern

- **Meaning:** Gateway handles common functionality instead of every microservice.
- **Example:** JWT validation, SSL termination, rate limiting and request logging happen at gateway level.

30. Consumer-Driven Contract Pattern

- **Meaning:** API consumers define/test expectations from provider APIs.
- **Example:** Cart Service expects Product Service always to provide `productId`, `name`, `price`.
- **Benefit:** Provider changes cannot silently break consumers.
- **Implementation:** Pact/Spring Cloud Contract.

---

2. Microservice Architecture Principles

1. Single Responsibility Principle

- **Rule:** One service should own one focused business responsibility.
- **Example:** Cart Service handles cart operations, not payment processing.

2. Separation of Concerns

- **Rule:** Separate unrelated technical/business responsibilities.
- **Example:** Cart logic stays in Cart Service; authentication belongs to security/identity components.

3. Loose Coupling

- **Rule:** Minimize dependencies between services.
- **Example:** Order Service integrates with Payment Service through an API/event contract, not Payment database tables.

4. High Cohesion

- **Rule:** Closely related functionality stays together.
- **Example:** Add Item, Remove Item, Update Quantity and Get Cart belong inside Cart Service.

5. Domain-Driven Design

- **Rule:** Model services around business domains.
- **Example:** Catalog, Cart, Order, Payment and Shipping represent separate business domains.

6. Bounded Context

- **Rule:** Give each domain clear model and ownership boundaries.
- **Example:** `"Product"` inside Catalog should not automatically become the data model used by Order Service.

7. Decentralized Data Management

- **Rule:** Each service controls its own data.
- **Example:** Order Service owns Order DB; Cart Service owns Cart DB.

8. API-First Design

- **Rule:** Design the service API before implementation.
- **Example:** Define `POST /carts/{id}/items` contract before coding Cart Service.
- **Implementation:** OpenAPI/Swagger.

9. Contract-First Design

- **Rule:** Agree on the formal integration contract first.
- **Example:** Order and Payment teams agree on Payment API/event schema before implementation.

10. Independent Deployability

- **Rule:** One service should be deployable without deploying unrelated services.
- **Example:** Deploy Cart Service v2 without redeploying Product Service.

11. Independent Scalability

- **Rule:** Scale services according to individual load.
- **Example:** During a sale, independently scale Cart and Product Services from 5 to 20 pods while Shipping remains unchanged.

12. Automation

- **Rule:** Automate repetitive engineering and operational processes.
- **Example:** Automatically build, test, scan and deploy Cart Service.

13. Infrastructure as Code

- **Rule:** Infrastructure should be version-controlled and reproducible.
- **Example:** Kubernetes resources/cloud infrastructure managed through Helm/Terraform.

14. Design for Failure

- **Rule:** Assume distributed components **will eventually fail**.
- **Example:** Cart Service is designed to tolerate temporary Pricing Service failure using timeout, retry or fallback behavior.

15. Fault Isolation

- **Rule:** Prevent failure from cascading across services.
- **Example:** Recommendation Service failure should not bring down checkout.

16. Fail Fast

- **Rule:** Detect invalid/unavailable conditions quickly.
- **Example:** Checkout immediately rejects an invalid Cart ID instead of performing unnecessary downstream processing.

17. Resilience

- **Rule:** Services should recover from temporary failures gracefully.
- **Example:** Payment calls use Timeout + Retry + Circuit Breaker where appropriate.
- **Implementation:** Resilience4j.

18. Idempotency

- **Rule:** Repeated requests should not accidentally duplicate business operations.
- **Example:** Retrying `Place Order` with the same idempotency key must not create multiple orders.

19. Eventual Consistency

- **Rule:** Distributed data may become consistent asynchronously rather than immediately.
- **Example:** An Order becomes `CONFIRMED`, while Analytics/Notification views update shortly afterward through events.

20. Observability

- **Rule:** Distributed systems require logs, metrics and traces.
- **Example:** Trace a checkout request through Gateway → Cart → Order → Payment → Inventory.
- **Implementation:** OpenTelemetry + centralized metrics/logging/tracing tooling.

21. Security by Design

- **Rule:** Security is built into architecture from the beginning.
- **Example:** Spring Security validates OAuth2/JWT access to Cart and Order APIs.

22. Zero Trust

- **Rule:** Never automatically trust a user/service merely because it is inside the network.
- **Example:** Payment Service authenticates/authorizes Order Service requests.

23. Statelessness

- **Rule:** Service instances should avoid storing client session state locally.
- **Example:** Any Cart Service pod can process the next request because cart state is in Redis/database, not local memory.

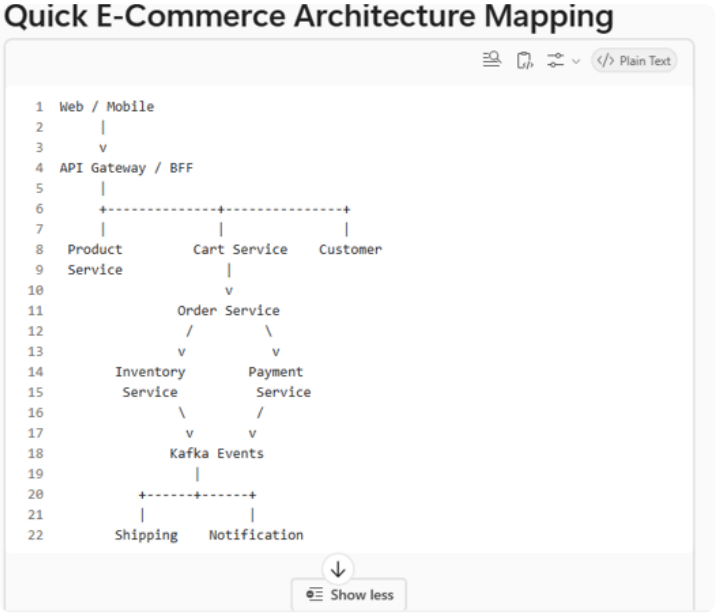
24. Backward Compatibility

- **Rule:** API changes should not unexpectedly break existing consumers.
- **Example:** Adding `discountAmount` should not break clients consuming an older Cart response.

25. Continuous Integration / Continuous Delivery

- **Rule:** Continuously build, test and make services releasable/deployable.
- **Example:** Cart Service commit triggers compilation → unit tests → integration/contract tests → security scan → Docker image → deployment pipeline.

Quick E-Commerce Architecture Mapping



Simple End-to-End Checkout Example

Customer Checkout

`Cart → Order → Inventory → Payment → Shipping → Notification`

Key patterns applied:

- **API Gateway** → receives checkout API request
- **BFF** → provides web/mobile-specific response
- **Database per Service** → each service owns data
- **Saga** → manages the distributed checkout transaction
- **Outbox** → reliably publishes Order events
- **Kafka/Event-Driven** → asynchronous communication
- **Circuit Breaker + Retry + Timeout** → resilience
- **Idempotency** → avoids duplicate payments/orders
- **DLQ** → handles repeatedly failing events
- **Observability** → traces the complete checkout flow
- **CI/CD + Kubernetes** → independently deploys and scales services

Interview shortcut: Think of the architecture as **DDD for service boundaries, API Gateway/BFF for client access, Saga + Outbox + Kafka for distributed transactions, Resilience4j for failures, Database-per-Service for autonomy, and Observability + CI/CD + Kubernetes for production operations**

End-to-End Interview Answer

> In an e-commerce platform, I divide the system into bounded contexts such as Catalog, Cart, Order, Inventory, Payment, and Shipping. Clients enter through an API Gateway or frontend-specific BFF. Each service owns its database and is independently deployable and scalable. During checkout, a Saga coordinates Order, Inventory, and Payment transactions. The Transactional Outbox and Kafka provide reliable event publication. Idempotency prevents duplicate orders and payments. Resilience4j supplies timeout, retry, circuit-breaker, and bulkhead controls. Spring Security protects APIs with OAuth 2.0 and JWT. Spring Boot Actuator, metrics, logs, and distributed tracing provide production observability.

Boundaries:

DDD + Bounded Context + Database per Service

Client Access:

API Gateway + BFF

Checkout Transaction:

Saga + Compensation

Reliable Messaging:

Transactional Outbox + Kafka + Idempotent Consumer + DLT

Failure Handling:

Timeout + Retry + Circuit Breaker + Bulkhead

Security:

OAuth 2.0 + JWT + Zero Trust

Operations:

Actuator + Logs + Metrics + Tracing + CI/CD + Kubernetes

This revision sheet condenses the uploaded Ritesh_Design_Patterns_List into interview-ready definitions, e-commerce examples, and selected Java/Spring Boot implementation snippets. The examples assume **Product, Cart, Inventory, Order, Payment, Shipping, and Notification microservices**. [Ritesh_Des..._List.docx | Word]

Microservices Interview Revision Sheet

1. Design Patterns

Pattern	Interview Definition	E-Commerce Example

API Gateway	Single entry point for client requests, routing, security, and rate limiting.	`/cart/**` routes to Cart Service; `/orders/**` routes to Order Service.
BFF	Dedicated backend designed for a specific frontend.	Web-BFF returns detailed cart data; Mobile-BFF returns a lightweight response.
Database per Service	Each service owns its database and exposes data through APIs/events.	Cart, Order, and Payment Services use separate databases.
Saga	Coordinates distributed local transactions with compensating actions.	Payment failure triggers order cancellation and inventory release.
CQRS	Separates write commands from read queries.	`AddItemToCart` updates data; `GetCartSummary` uses an optimized read model.
Event Sourcing	Stores state changes as events.	Cart state is rebuilt from `ItemAdded`, `QuantityChanged`, and `ItemRemoved`.
Circuit Breaker	Temporarily stops calls to an unhealthy service.	Cart Service stops calling an unavailable Recommendation Service.
Retry	Retries transient failures with controlled backoff.	Order Service retries a temporary Inventory Service failure.
Timeout	Limits how long a service waits for a response.	Checkout waits a maximum of two seconds for Pricing Service.
Bulkhead	Isolates resources to prevent cascading failure.	Payment and Recommendation calls use separate thread pools.
Service Discovery	Locates service instances dynamically.	Kubernetes DNS resolves `inventory-service`.
Externalized Configuration	Keeps environment configuration outside application code.	Payment URLs and timeouts come from ConfigMaps and Secrets.

Sidecar	Runs infrastructure functionality beside the application.	An Envoy sidecar provides proxying and telemetry.
Strangler Fig	Replaces a monolith incrementally.	Cart is extracted first, followed by Order and Payment.
Anti-Corruption Layer	Translates an external or legacy model into an internal domain model.	Legacy payment codes are converted into `PaymentStatus`.
Aggregator	A dedicated component combines multiple service responses.	Cart Aggregator combines cart, product, price, and stock information.
API Composition	Builds a result by querying multiple services.	Order Details combines Order, Payment, and Shipping responses.
Transactional Outbox	Saves business data and an event record in one local transaction.	Order and `OrderCreated` outbox record are saved together.
Event-Driven Architecture	Services communicate through asynchronous business events.	Order Service publishes `OrderCreated` to Kafka.
Publish-Subscribe	Multiple subscribers independently consume the same event.	Inventory, Notification, and Analytics consume `OrderCreated`.
Competing Consumers	Multiple instances process messages from the same queue/topic group.	Notification Service instances process order events in parallel.
Idempotent Consumer	Reprocessing the same event does not duplicate the result.	Duplicate `PaymentCompleted` events create only one shipment.
Dead Letter Queue	Stores repeatedly failing messages for investigation.	Invalid order events move to an Order DLT after retry exhaustion.
Health Check	Reports whether a service is alive and ready.	Kubernetes checks Spring Boot Actuator readiness.

Ambassador	External proxy handles outbound communication concerns.	A proxy secures and retries calls to a payment provider.
Service Mesh	Infrastructure layer manages service-to-service networking.	Istio provides mTLS, traffic control, metrics, and tracing.
Gateway Routing	Gateway routes paths to the appropriate services.	`/api/products/**` routes to Product Service.
Gateway Aggregation	Gateway combines results from multiple services.	`/checkout-summary` combines Cart, Inventory, Discount, and Shipping.
Gateway Offloading	Gateway handles shared technical concerns.	JWT validation, SSL termination, CORS, and rate limiting occur at the gateway.
Consumer-Driven Contract	Consumers define expectations that providers continuously verify.	Product Service changes are tested against Cart Service expectations.

---

2. Architecture Principles

Principle	Concise Meaning	E-Commerce Application
Single Responsibility	One service owns one focused responsibility.	Cart Service manages carts, not payments.
Separation of Concerns	Separate business, security, persistence, and infrastructure responsibilities.	Authentication is not implemented inside cart business logic.
Loose Coupling	Services depend on contracts, not internal implementation.	Order Service uses Payment API/events, not Payment DB tables.
High Cohesion	Related functionality remains within the same service.	Add, update, remove, and view cart items belong to Cart Service.

Domain-Driven Design	Model architecture around business domains.	Catalog, Cart, Order, Payment, and Shipping are separate domains.
Bounded Context	Each domain has clear model and ownership boundaries.	`Product` in Catalog differs from the Order Service product snapshot.
Decentralized Data	Services independently own and manage their data.	Each service has a separate schema or database.
API-First	Design the API before implementing it.	Define the Cart API using OpenAPI before Java development.
Contract-First	Agree on formal request, response, and event contracts first.	Order and Payment teams agree on the payment contract.
Independent Deployability	Deploy one service without deploying unrelated services.	Cart Service v2 is released without rebuilding Product Service.
Independent Scalability	Scale services according to individual workload.	Cart Service scales to 20 pods during a major sale.
Automation	Automate build, testing, scanning, and deployment.	A GitHub Actions or Azure DevOps pipeline deploys each service.
Infrastructure as Code	Define infrastructure in version-controlled code.	Terraform and Helm define cloud and Kubernetes resources.
Design for Failure	Assume dependencies and networks can fail.	Cart Service has timeout and fallback behavior for Pricing Service.
Fault Isolation	Contain failure within the affected component.	Recommendation failure does not block checkout.
Fail Fast	Reject invalid operations immediately.	Checkout immediately rejects an empty or expired cart.

Resilience	Recover gracefully from temporary failure.	Use timeout, retry, circuit breaker, and fallback.
Idempotency	Repeated requests produce one business result.	Repeated checkout requests with the same key create one order.
Eventual Consistency	Distributed data becomes consistent asynchronously.	Payment succeeds first; order status updates through an event shortly afterward.
Observability	Use logs, metrics, traces, and alerts.	A trace follows Gateway → Cart → Order → Payment.
Security by Design	Build authentication, authorization, and protection from the start.	APIs use OAuth 2.0, JWT, validation, and least privilege.
Zero Trust	Authenticate and authorize every user and service request.	Payment Service validates the identity and permissions of Order Service.
Statelessness	Do not keep client session state inside a service instance.	Cart state is stored in Redis or a database, not pod memory.
Backward Compatibility	Existing consumers continue working after API evolution.	A new optional field is added without removing existing fields.
CI/CD	Continuously build, test, release, and deploy services.	Each Cart Service commit triggers tests, scanning, image creation, and deployment.